# CHAPITRE XI

# GUI: LES INTERFACES GRAPHIQUES (JavaFx)

# HISTORIQUE(Chronologie)

| 1995 | 1996 | 1998 | 2007 | 2011 | 2014-2018 |
|---|---|---|---|---|---|
| AWT | JAVA1.0 SWING | JAVA1.2 | Annonce de JavaFX | Java7 | Java8-JavaFX |

A l'origine du langage Java, les interfaces graphiques étaient créées en utilisant la librairie AWT (java.awt)

Rapidement, la librairie Swing (javax.swing) est venu compléter (et partiellement remplacer) la librairie AWT

# HISTORIQUE(SWING vs AWT)

- Tous les composants du package AWT sont des composants lourds.
- Les composants Swing (nom du package : javax.swing) sont pour la majorité d'entre eux des composants légers.
- Tous les composants Swing commencent par la lettre " J ".
- Les composants Swing se répartissent :
  - en conteneurs de plus lourd:
    - **JFrame** à rapprocher de la classe **Frame** dans AWT
    - **JDialog** à rapprocher de la classe **Dialog** dans AWT
    - **JWindow** à rapprocher de la classe **Window** dans AWT
    - **JApplet** à rapprocher de la classe **Applet** dans AWT
  - en conteneurs légers (les autres contrôles GUI Swing).

# HISTORIQUE(SWING vs AWT)

| JavaFX 1 | JavaFX 2 | JavaFX 8 |
|---|---|---|
| a tenté, sans grand succès, de remplacer Swing | Pas de script.<br>Java API et/ou XML:FXML SceneBuilder, CSS | Plus de librairie externe<br>à télécharger et à référencer<br>**Scene Builder 2 , lambda**<br>et les streams ,DatePicker<br>TreeTableView.<br>Gestion des écrans tactiles<br>(TouchEvent,GestureEvent,)<br>Amélioration<br>des librairies graphiques 2D et 3D ,CSS |

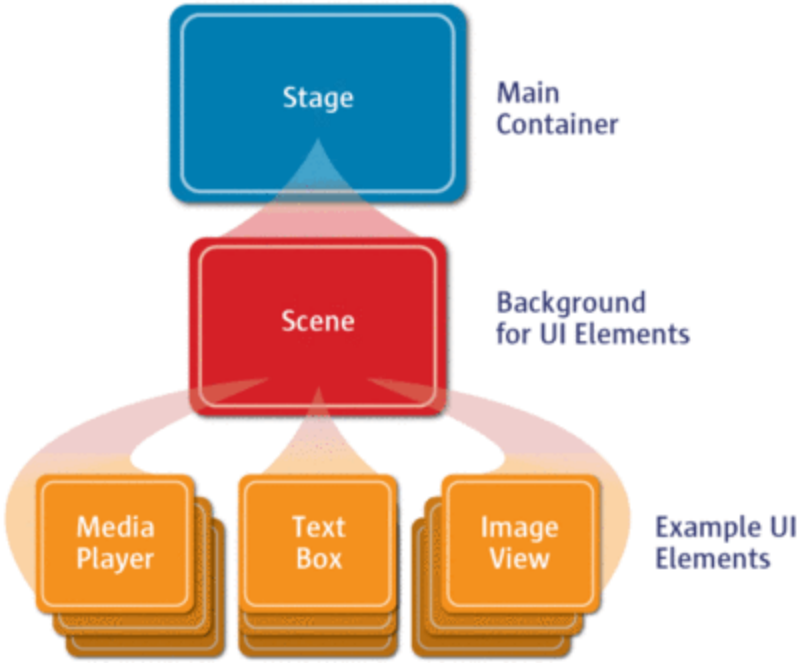
# Presentation d'une application JavaFX

Pour construire une application JavaFX il faut:

- ➢Hériter de la classe Application.

- ➢Créer un objet de type **Stage** pour le lancement de l'application.

- ➢Créer un objet de type **Scene** et l'associer à la fenêtre(Stage).

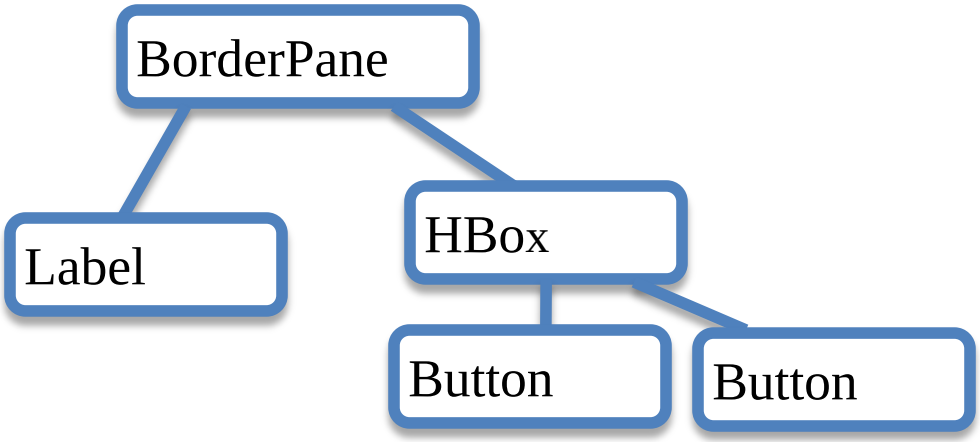



- ➢La Scène est composée de nœuds (**Node**)qui représente les différents éléments de l'interface graphique.

- ➢Enfin on appel la méthode start() qui construit l'interface.

# Graphe de Scene

| Forme primitive (**Shape**) | Conteneurs(**Layout-Pane**) | Composant standard (**Control**) | Composants spécialisés |
|---|---|---|---|
| Line, Circle,Rectangle, Box, Cylinder | AnchorPane, StackPane, BorderPane, GridPane, HBox, Vbox, | Label, Button, TetField, ComboBox | MediaView, WebView, ImageView, |



# Creation d'une fenetre personnalisee

```
public class PremiereFenetre extends Application {
Button bouton;
public void start(Stage premierStage)
{
    premierStage.setTitle("Ma premiere application javaFx");
    bouton=new Button();
    bouton.setText("Click me");
    StackPane root=new StackPane();
    root.getChildren().add(bouton);
    Scene scene=new Scene(root,300,250);
    premierStage.setScene(scene);
    premierStage.show();
}
public static void main(String[] args) {
launch(args);}}
```

# Action sur un Bouton

```
public class PremiereFenetre extends Application
implements EventHandler<ActionEvent>{
Button bouton;
public void start(Stage premierStage)
{
        premierStage.setTitle("Ma premiere application javaFx");
        bouton=new Button();
        bouton.setText("Click me");
        StackPane root=new StackPane();
        root.getChildren().add(bouton);
        Scene scene=new Scene(root,300,250);
        bouton.setOnAction(this);
        premierStage.setScene(scene);
        premierStage.show();}
        public static void main(String[] args) {
        launch(args);}
public void handle(ActionEvent event) {
if(event.getSource()==bouton)
System.out.println("Vous m'avez touche");}}
```

La méthode **gerSource** fournit une référence sur l'objet ayant déclenché l'événement concerné.

# Classe Anonyme

Il est possible de définir une classe interne, sans lui donner de nom par dérivation d'une super classe, ou par implémentation d'une interface.

```
bouton.setOnAction(new
EventHandler<ActionEvent>(){
public void handle(ActionEvent event) {
if(event.getSource()==bouton)
System.out.println("Vous m'avez touche"); }});
```

# Les expressions Lamda

```
bouton.setOnAction(e->System.out.println("Vous m'avez touche"));
```

Les expressions **lambd**a sont aussi nommées **closures** ou fonctions anonymes : leur but principal est de permettre de passer en paramètre un ensemble de traitements.

# Les événements clavier

- **onKeyPressed:** touche enfoncée
- **onKeyReleased:** touche relâchée
- **onKeyTyped:** touche saisie

Pour recevoir des événements il faut obtenir le focus. On doit mettre à true le focusTransversable

# Les événements clavier

```
public class EvenementClavierSouris extends Application{
//Button bouton;
public void start(Stage premierStage)
{
Circle cercle=new Circle();
cercle.setCenterX(100);
cercle.setCenterY(100);
cercle.setRadius(50);
cercle.setFill(Color.AQUAMARINE);
cercle.setFocusTraversable(true);
cercle.setOnKeyPressed(KeyEvent->System.out.prinln("Touche enfoncee: :"+KeyEvent.getCode()+" "
+KeyEvent.getCharacter()));
cercle.setOnKeyReleased(KeyEvent->System.out.println("Touche relachee:"+KeyEvent.getCode()+" "
+KeyEvent.getCharacter()));
cercle.setOnKeyTyped(KeyEvent->System.out.println("touche saisie: "+KeyEvent.getCode()+ " "
+KeyEvent.getCharacter()));
premierStage.setTitle("Evenement clavier");
Pane root=new Pane();
root.getChildren().setAll(cercle);
Scene scene=new Scene(root,300,250,Color.GREEN);
premierStage.setScene(scene);
premierStage.show();}
public static void main(String[] args) {
launch(args);}}
```

# Les événements souris

- **onMouseClicked:** clic de souris
- **onMouseDragged:** la souris cliquée-tiree
- **onMouseEntered:** la souris est entrée dans la zone du nœud
- **onMouseExited:** la souris est sortie de la zone
- **onMouseMoved:** la souris est deplacee
- **onMousePressed:** la souris est enfoncée
- **onMousereleased:** la souris est relâchée

On n'a pas besoin de mettre à true le focusTransversable pour obtenir le focus.

# Les événements souris

cercle.setOnMouseClicked(MouseEvent->System.***out.println("Vous avez clique sur le point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

cercle.setOnMouseDragged(MouseEvent->System.***out.println("Vous avez effectue un drag'n drop sur le point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

cercle.setOnMouseEntered(MouseEvent->System.***out.println("Vous avez entre la souris sur le point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

cercle.setOnMouseExited(MouseEvent->System.***out.println("Vous avez sorti la souris au point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

cercle.setOnMouseMoved(MouseEvent->System.***out.println("Vous avez deplace sur le point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

cercle.setOnMousePressed(MouseEvent->System.***out.println("Vous avez appuye la souris sur le point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

cercle.setOnMouseClicked(MouseEvent->System.***out.println("Vous avez relache la souris au point de coordonnees: "+MouseEvent.getX()+" "+MouseEvent.getY()));***

# Naviguer entre deux scenes

```
public class Mafenetre extends Application
{Stage window;
Scene scene1,scene2;
public static void main(String args[])
{launch(args);}
public void start(Stage primaryStage) throws Exception {
//****** Initialisation et instanciation ****************
window=primaryStage;
Label label = new Label("Bienvenue sur la premiere Scene");
Button bouton1 = new Button("premiere Scene");
Button bouton2 = new Button("Deuxieme Scene");
VBox vb=new VBox(20);
StackPane stp= new StackPane();
scene1=new Scene(vb,300,200);
//******************** Ajout de composant ***************************
vb.getChildren().addAll(bouton1,label);
stp.getChildren().add(bouton2);
scene2=new Scene(stp,500,300,Color.RED);
window.setScene(scene2);
window.setTitle("Naviger");
//******************** Action ***************************
bouton1.setOnAction(e->window.setScene(scene2));
bouton2.setOnAction(e->window.setScene(scene1));
window.show();}}
```

**Label:** Pour afficher du texte dans un conteneur.

**VBox** est un conteneur qui arrange les sous-composants sur une seule colonne

# Layout-Panes

Les layout permettent aux utilisateurs d'avoir un large choix sur la structure et la position qu'il faut adopter pour les composants qui seront placés sur la fenêtre.

| | |
|---|---|
| Hbox-VBox | Les composants sont placés horizontalement ou verticalement. |
| FlowPane(Horizontal-Vertical) | Les composants sont placés horizontalement/verticalement avant de passer à la ligne/colonne suivante s'il n'ya plus d'espace. |
| BorderPane | Les composants peuvent être placés dans cinq emplacements différents: Top, Bottom, Left, Right, Center. |
| AnchorPane | Place les composants en respectant une contrainte de distance par rapport à un ou plusieurs bords du conteneur. |
| StackPane | Place les composants les uns au dessus des autres |
| GridPane | Place les composants dans une grille |

# Couleur de bordures et d'arrière-plans

```
Background bgGreen = new Background( new BackgroundFill(Color.LIGHTGREEN,
new CornerRadii(20), new Insets(15) ));
hb.setBackground(bgGreen);
```

```
Background bgImg = new Background( new BackgroundImage( img,null,null,
BackgroundPosition.CENTER, null));
hb.setBackground(bgImg);
```

# Les composants(Les etiquettes et les champs de texte)

Pour afficher du texte dans un conteneur on utilise la classe **Label:**

**Label lb=new Label("Factoriel");**

Le texte d'une étiquette peut être modifier avec la méthode **setText()**

Un champ de texte est une zone rectangulaire dans laquelle l'utilisateur peut entrer ou modifier des valeurs. On utilise la classe **TextField** pour instancier un tel objet.

**TextField tf=new TetField();**

**tf.setPromptText (" valeur à calculer ");// valeur par défaut**

L'information figurant dans un champs texte s'obtient avec la méthode **getText().**

**PasswordField** permet de saisir un texte sans que celui-ci soit affiché dans le champ texte.

Pour utiliser une zone de texte multilignes on utilise la classe **TextArea.**

**TextArea txa = new TextArea();**

**txa. setPrefRowCount(8);**

# Les composants(Bouton radio)

On utilise la classe **RadioButton** pour les boutons radio.

```
ToggleGroup tg = new ToggleGroup();
 on.setToggleGroup(tg);
off.setToggleGroup(tg);
```

**Les actions sur les boutons radio:**

```
tg.selectedToggleProperty().addListener(new ChangeListener<Toggle>() {
        @Override
        public void changed(ObservableValue<? extends Toggle> ov, Toggle old_toggle,
Toggle new_toggle) {

            if (tg.getSelectedToggle() != null) {
                RadioButton button = (RadioButton) tg.getSelectedToggle();
                System.out.println("Bouton: " + button.getText());
                }} });
```

# Les composants(les cases à cocher)

- Les cases à cocher permettent à l'utilisateur d'effectuer un choix de type oui/non. Elle est instancier grâce à la classe CheckBox.
- CheckBox cb=new CheckBox();

## Les actions sur les cases à cocher:

```
CheckBox cb=new CheckBox(« marié");
   cb.setOnAction((event) -> {
  if(cb.isSelected())
   System.out.println("la case "+cb.getText()+" est selectionnée");
});
```

# Les composants(Hyperlink)

- Hyperlink est un composant semblable à un lien hypertexte HTML dans une page web.

```
Hyperlink hl=new Hyperlink("Site de l'UADB");
    hl.setOnAction(event-> { getHostServices().showDocument("www.uadb.edu.sn"); });
```

**Tooltip** peut être ajouté à n'importe quel composant:

```
Tooltip    tt = new Tooltip("Cliquez pour choisir une marque de machine");
    bouton1.setTooltip(tt);
```